EasyNet Ability Owner — Truth Table

AXIOM Invocation envelope projection for the default catalogue

---

Author: 凉冰 (architect) Date: 2026-05-05 Status: SPECIFICATION

Why this document exists

This is a flat-fact reference: a single page-pair that answers, for every default-registered ability, three questions:

1. *What URA names its owner* (the `owner_agent_uri` field on its descriptor).
2. *Where it lands in the AXIOM Invocation envelope* (`caller` / `callee` / `subject` slots).
3. *Why EasyNet ships it by default* (the protocol assumption that breaks if you delete it).

The CLI render layer, `meta.list_abilities` synth path, and federation advertise pipeline all derive owner mapping independently. The truth table is the contract those derivations must agree with.

1 URA grammar (AXON-RFC-001 v4.1.5 §A.URA)

Every value referenced in this document conforms to the v4.1.5 URA grammar. The grammar is closed: the parser (`src/uri.rs::parse\_ura`) rejects any string outside the six shapes below. Other "URA-shaped" tokens that have appeared in practice (`<self>`, `<unjoined>`, `agent://self`) are *not* URAs and must not propagate past the registration boundary.

Ontology

Relation	Meaning
realm *owns* hub	A realm is identified by its hub URA; one hub per realm.
user *owns* agent	Ownership stays with the user across device migrations.
device *hosts* agent	Device is the runtime placement, not the identity.
agent *exposes* ability	Verbs are mounted under their owning agent.
resource *is* user-owned concrete substrate	Files, processes, ptys, sockets —typed by namespace.

The six URA shapes

Kind	Wire shape	Tail invariants
`user`	`easynet:///r/<realm>/user/<user-uuid>`	user-uuid is bare (no `.`, no `/`)
`device`	`easynet:///r/<realm>/device/<device-uuid>`	device-uuid is bare
`agent`	`easynet:///r/<realm>/agent/<user-uuid>.<agent-id>`	exactly one `.` in tail; both segments non-empty; agent-id has no `.`
`ability`	`easynet:///r/<realm>/ability/<user-uuid>.<agent-id>.<ability-id>`	exactly two leading `.`s; ability-id may itself contain `.`s (e.g. `fs.read`)

hub	easynet:///r/<realm>/hub	realm-singleton; no tail; no sub-id (the legacy /hub/01HUB tail is retired)
resource	easynet:///r/<realm>/resource/<user-uuid>/<ns>/<path>	<ns> ∈ {fs, process, pty, shell, http}; v4.1.5 also accepts resource/<user> .<owner-tail> /<path> for content-addressed owners

Three properties used throughout this document

P1 — URAs are self-describing
Any URA, in isolation, names exactly one actor in the seven-tuple ontology. Nothing about who *calls* the URA changes what it names. A string that requires caller context to resolve (<self>, the literal placeholder self, an agent://self URN) is therefore not a URA, and must not appear in any persisted descriptor, advertise payload, or receipt field.

P2 — Owner is named at registration, not derived from the name
The descriptor's owner_agent_uri is the wire form of "who advertises this ability". It is fixed at register_rpc time, by the registration site, against an explicit owner URA. It is *not* inferred from name.starts_with(…) prefix sniffing. Every consumer reads the stored value verbatim.

P3 — callee URA carries the actor, not the verb
The ability URA kind exists only at the publish layer (descriptor, advertise). It never appears in an Invocation envelope's callee slot. Per AXON-RFC-001 v4.1.5 §9 (AXIOM seven-tuple), callee ∈ {hub, device, agent}. The verb itself is in function_name, which is a string, not a URA.

2 The AXIOM Invocation envelope

Per AXON-RFC-001 v4.1.5 §9 the Invocation envelope carries seven URA-typed bindings; only the four that govern owner mapping are reproduced here.

Field	Allowed URA kind	Semantics
caller	hub / device / agent	Who initiates the call. Required by Invariant 1.
callee	hub / device / agent	Which actor runs the handler. *Never* ability or user.
subject	any URA kind, may equal callee	Resource the call acts on. None is allowed only for meta-style abilities (INV-META-SUBJECT-EXEMPT); resource handlers must reject None.
function_name	string, not a URA	The ability name (device.fs.read, <owner>.chat, hub.openai.chat_completions, …).

3 Truth table: name shape → owner kind

Every default-registered ability falls into one of the following name shapes. For each shape the table fixes the canonical owner_agent_uri, the resulting envelope.callee, and the default envelope.subject convention. The *Where* column points to the registration call site, which is the authoritative source for the owner per P2.

#	Name shape	Canonical owner URA	Default `subject`	Where
1	`device.fs.*`, `device.http.*`, `device.shell.*`, `device.process.*`	`device/<id>`	resource URA, or `=callee` for meta verbs	AXIOM Tier 2.5 baseline-locomotion-v1 profile
2	`device.camera.*`, `device.mic.*`, `device.screen.*`, `device.speaker.*`	`device/<id>`	*required* resource URA; handler rejects `None`	`profiles::device` (media)
3	`device.fleet.*`, `device.observe.*`, `device.admin.*`, `device.discuss.*`, `device.loop.*`, `device.schedule.*`, `device.mission.*`, `device.easynet.*`, `device.meta.*`, `device.ability.*`, `device.policy.*`, `device.consent.*`, `device.skill.*`, `device.mcp.*`, `device.a2a.*`, `device.voice.*`	`device/<id>`	`=callee` (meta)	`profiles::device`; runtime `register_rpc`
4	`<agent>.discover`, `<agent>.invoke`, `<agent>.api_key.*` (one row per mounted sub-agent; `<agent>` ∈ {`claude`, `codex`, `web-builder`, …})	`agent/<u>.<agent>`	`=callee` (meta)	`discover_ability::register_for_agent` `invoke_ability::register_for_agent` `api_key_ability::register`
5	`device.keyring.*`, `device.session`, `device.register_device_pubkey`	`device/<id>`	`=callee` (RFC-002 §3.3)	`keyring::abilities::register_for_owner` `axon_serve::admission_facade`
6	`<agent>.chat`	`agent/<u>.<agent>`	`=callee` (chat is meta)	`chat_ability::register`
7	`<agent>.<custom-verb>` (e.g. `web-builder.todo_add_task`)	`agent/<u>.<agent>`	task or resource URA	per-agent profile `register`
8	`consent.*`, `policy.*` (handler form), `mcp.bridge.*`, `mcp.client.*`, `a2a.bridge.*`, `a2a.client.*`, `voice.*` (handler form), `skill.*`	`agent/<u>.<helper-id>` for hosted helper agents (e.g. `consent-default-0`); `device/<id>` for daemon-side adapters	varies; resource URAs for outbound adapters	`profiles::consent`; `profiles::policy`; `profiles::mcp`; `voice` module
9	`hub.openai.*`, `hub.<namespace>.*`	`hub` (`easynet:///r/<r>/hub`, realm-singleton)	`=callee` or per-verb (e.g. `chat_completions` carries no subject)	`openai_compat_ability::register`
10	`<user>.pages.*`, `<user>.<project>.page.fetch`, `<user>.files.*`	`agent/<u>.<owner-tail>` or `user/<u>` depending on profile	resource URA (`resource/<u>.<owner-tail>/...`)	`pages::register`; `files::register`

#	Name shape	Canonical owner URA	Default `subject`	Where
11	`<user>.api_key.*` (`<user>` is the canonical user-id from credentials, not a placeholder)	`user/<user>`	`=callee` (meta)	`api_key_ability:: register`

Notes on the table

Row 4 is per-sub-agent
The catalogue contains one `discover/invoke/api_key.*` row *per mounted sub-agent*. With sub-agents `claude` and `codex` on the same device, the catalogue carries `claude.discover+codex.discover+…`, each with its own owner URA. There is no shared "self" entry.

Row 5 is exactly one row
Daemon-bundle abilities (`device.keyring.*`, `device.session`, `device.register_device_pubkey`) are mounted once per daemon, with `owner_agent_uri = device URA`. They are not per-agent.

Row 8 has two sub-cases
An adapter such as `consent.decide` can be hosted either as a sub-agent (`consent-default-0`, owner = `agent/<u>.consent-default-0`) or as a daemon-side bridge (`mcp.bridge.list_tools`, owner = `device/<id>`). The catalogue records the sub-case at registration time — see P2.

Row 9 owner is the realm hub
`hub.*` handlers physically run on the device daemon (the hub forwards via federation), but the ability is conceptually a hub-tier projection. The descriptor must say so. Synth derives the owner URA via `hub_uri(credentials.tenant_id)` — not from the device URA.

Row 11 `<user>` is canonical
The `<user>` slot is the canonical user-id from `credentials.username`; placeholder defaults are not part of the spec.

4 CLI render projection

The `easynet ability list` table projects the `owner_agent_uri` into the columns DEVICE / AGENT / USER. Each column reflects a slot of the owner URA's kind; columns the owner does not name stay as a literal dash. Mixing kinds in one row (the `device=99e59cc... agent=hub` bug from this PR's audit) is a renderer-level violation of P3 and must be rejected by tests.

Owner URA kind	DEVICE	AGENT	USER	Notes
`device/<id>`	`<id>`	-	-	rows 1, 2, 3, 5
`agent/<u>.<a>`	-	`<a>`	`<u>`	rows 4, 6, 7, 8a, 10a
`hub`	-	`hub`	-	row 9
`user/<u>`	-	-	`<u>`	row 11 (canonical)
parse failure / unknown	-	`!err`	-	catalogue corruption signal; do *not* silently dash

5 Default catalogue — abilities and why each exists

The 121 abilities below are what `easynet ability list` returns on a freshly-paired daemon (realm `device.easynet.run`, user `test`, device `99e59ccd-...debf73d`, with one hosted sub-agent under the `web-builder` profile). For each ability we state *why EasyNet must ship it by default* — the protocol assumption that breaks if you delete the row.

5.1 Hub-owned (owner = `hub`)

`hub.openai.chat_completions`, `hub.openai.list_models`. RFC-006-C v0.1 OpenAI compatibility surface. Without these, an OpenAI SDK pointed at an EasyNet daemon cannot do non-streaming chat or model enumeration — the entire "drop-in OpenAI replacement" story fails at the wire. The verbs are hub-rooted (not device-rooted) because the realm hub is the federation-visible endpoint that external SDKs dial; the device daemon executes as the hub's local-execution proxy. The legacy on-wire form is `01HUB.openai.*`; the rename to `hub.*` ships with RFC-001 v4.1.6.

5.2 Device-owned — AXIOM Tier 2.5 baseline-locomotion-v1 (under `device.*`)

These four namespaces are the AXIOM *baseline locomotion* profile. Every Tier-2.5-conformant device ships them; the 8-stage destructive- intent / argv-parsing / receipt-redaction pipeline they share is the *reason* EasyNet can claim "an LLM can move on this substrate without bypassing audit". Removing any of them collapses the audit contract.

`device.fs.edit`, `device.fs.list`, `device.fs.read`, `device.fs.write`
Filesystem locomotion. `device.fs.edit` is the surgical string-replace primitive that lets agents diff-apply rather than full-rewrite (audit-trail compactness).

`device.http.request`
One outbound HTTP request per invocation. The receipt captures status, headers, body bytes for the audit pipeline; without this primitive every web-fetching agent has to escape into shell and the audit trail is forfeit.

`device.shell.run`
Bash command string with the full AXIOM Tier-2.5 8-stage security pipeline. The destructive-intent classifier and shell tokeniser live here.

`device.process.exec`
OS-level argv (no shell interpretation) — the surface for tools that need predictable argument passing without the shell metacharacter risk surface. Pairs with `device.shell.run` as the safe-default counterpart.

5.3 Device-owned — media resources (under `device.*`, subject required)

The four media namespaces project hardware resources into the ability surface. They are the *only* way an agent on this device can see/hear/speak; no fallback path exists. Subject (a resource URA naming the camera/mic/screen/speaker handle) is mandatory — handlers return `resource_not_found` on `None` per INV-SUBJECT-ENVELOPE.

`device.camera.snapshot`, `device.camera.subscribe`
Single still vs. pushed video stream from a camera resource.

`device.mic.subscribe`
Pushed audio stream from a microphone resource.

`device.screen.snapshot`, `device.screen.subscribe`
Single still vs. pushed video stream of a display target.

`device.speaker.publish`
Outbound audio frames to a speaker resource. Companion to `device.voice.subscribe` (TTS).

5.4 Device-owned — fleet operational surface (under `device.fleet.*`)

The fleet namespace exposes *cross-device operator verbs through the same Invoke surface as everything else.* Without them, operators have to ssh into each device — which forfeits AXIOM auditability. Per RFC §18 the device profile owns these.

Node lifecycle:
`device.fleet.list_nodes`, `device.fleet.describe_node`, `device.fleet.register_self`, `device.fleet.deregister_self`, `device.fleet.remove_node`. Operator-facing fleet topology view.

Ability deployment:
`device.fleet.deploy_ability`, `device.fleet.uninstall_ability`, `device.fleet.list_abilities`. Cross-device ability publish + uninstall via the Invoke surface (mirrors `easynet ability deploy`).

Agent lifecycle:
`device.fleet.start_agent`, `device.fleet.stop_agent`, `device.fleet.list_agents`. Sub-agent registry mutations through Invoke (mirrors `easynet agent add` / `remove`).

Skill management:
`device.fleet.skill_install`, `device.fleet.skill_remove`, `device.fleet.skill_upgrade`. Marketplace skill ops on a target device's `<agent-root>/skills/` dir.

Sessions (interactive PTY):
`device.fleet.session_create`, `device.fleet.session_attach`, `device.fleet.session_input`, `device.fleet.session_read`, `device.fleet.session_resize`, `device.fleet.session_close`, `device.fleet.list_sessions`, `device.fleet.attach_session`. PTY-aware interactive shell over Invoke; pairs with `device.fleet.exec_remote` (one-shot) for non-interactive cases. The `pty_session_*` variants are explicit aliases retained for legacy callers.

Direct ops:
`device.fleet.exec_remote`, `device.fleet.file_transfer`. One-shot remote command + bidirectional file copy. Without these the operator has no way to land a file or check a status without out-of-band access.

5.5 Device-owned — observability, admin, discovery aliases (under `device.*`)

`device.admin.status`
Operator-facing component snapshot (build version, membership state, ability-registry size). Pairs with the two `observe.*` verbs as the daemon's three-tier health surface.

`device.describe`
Device-profile self-description. Cross-device addressing helper (lets a peer ask "are you who I think you are" without pulling the full ability catalogue).

`device.observe.health`
Round-trip smoke ability. Returns the caller's args plus a daemon timestamp. The minimal proof of liveness; every CI sanity check uses this.

`device.observe.network_health`
Live network posture: local reachability, realm-directory reachability, peer-hub status. The diagnostic that distinguishes "daemon down" from "daemon up but hub unreachable".

`device.easynet.discover`, `device.easynet.run`, `device.easynet.cancel`, `device.easynet.track`
User-facing aliases that LLM prompts read more naturally than the literal `meta.list_abilities` / `device.mission.run` / `device.loop.cancel` / `device.loop.status`. Both names route to the same handler so the catalogue and dispatch table stay in lockstep.

5.6 Device-owned — mission, scheduling, loops, discuss (under `device.*`)

`device.mission.run`, `device.mission.think`, `device.mission.discuss_round`
EAL mission execution exposed as Invoke. Without these an LLM inside an agent has no first-class way to compose multi-step / multi-agent workflows — it would have to shell out to the `easynet mission run` CLI, which forfeits in-process state. `device.mission.discuss_round` drives one round of multi-agent conversation; the underlying state shares the `discuss.*` room model.

`device.schedule.add`, `device.schedule.list`, `device.schedule.enable`, `device.schedule.remove`
Cron-style schedule registry over Invoke. The daemon's tick runner reads from the same store. Without scheduling-as-an-ability every recurring task would need an external cron and would skip the AXIOM receipt trail.

`device.loop.create`, `device.loop.status`, `device.loop.subscribe`, `device.loop.cancel`
Worker+verify loop primitive. The `Live` stream variant surfaces per-iteration frames; this is what powers "run-until-passes-verifier" compositions without polling.

`device.discuss.create`, `device.discuss.post`, `device.discuss.list_turns`, `device.discuss.subscribe`
: Multi-agent discussion room. The append-only turn log + the `SnapshotThenLive` stream pattern are how a UI joining mid-conversation gets past turns + new ones in one call. `device.mission.discuss_round` writes into this same store.

5.7 Device-owned — edge adapters and policy (under `device.*`)

`device.a2a.bridge.list_skills`, `device.a2a.bridge.send_task`
: Inbound A2A: surface this device's local A2A agent cards to external A2A clients. Without the bridge, the realm's A2A catalogue is invisible to A2A-native clients.

`device.a2a.client.send_task`
: Outbound A2A: dispatch an RPC against a remote A2A agent. The other half of the agent-to-agent protocol bridge.

`device.mcp.bridge.list_tools`, `device.mcp.bridge.call_tool`
: Inbound MCP: project the local ability registry as MCP tools for an MCP-native client at the edge. Per "MCP at the edge, not node-to-node" (RFC §A3) this lives on the device, not inside the federation.

`device.mcp.client.call`, `device.mcp.client.list`
: Outbound MCP: dial a remote MCP server, list and call its tools. Pairs with the bridge.

`device.policy.evaluate`, `device.policy.simulate`
: RFC §A6 admission policy. The kernel admission gate calls `device.policy.evaluate` as an in-process sub-invocation carrying the original envelope; `device.policy.simulate` is the operator-facing dry-run. Without these the daemon has no policy hooks at all and every Invoke is admitted unchecked.

5.8 Device-owned — ability publishing primitives (under `device.ability.*`)

`device.ability.publish`, `device.ability.unpublish`
: Publish-layer verbs that push a descriptor through the federation advertise pipeline. Without them the only way to add a new ability is to redeploy the daemon, which forfeits the live-update story.

5.9 Device-owned — daemon device-bundle (row 5)

These are RFC-002 §3.3 abilities the daemon hosts directly, with no sub-agent wrapper. The catalogue carries exactly one row each.

`device.keyring.create`, `device.keyring.list`, `device.keyring.get_public`, `device.keyring.sign`, `device.keyring.rotate`, `device.keyring.revoke`, `device.keyring.expire_set`, `device.keyring.bind_subject`, `device.keyring.peer_add`, `device.keyring.peer_list`, `device.keyring.federate_user_identity_token`
: RFC-002 keyring: the device-scoped private-key store + signing primitive + cross-realm federated identity token issuance. Without it, no AXIOM Invoke can be signed and federation breaks at the signing-authority check.

`device.session`
: The long-lived bidi RPC session the device dials to its realm hub on boot. All hub→device forward-Invokes ride this session. Without it the hub has no way to push work down to the device.

`device.register_device_pubkey`
: Pairing-flow primitive: writes the device's Ed25519 public key into the hub's realm trust anchor. Without it, the hub's admission gate would reject every signed Invoke from the device.

5.10 Agent-owned (per-sub-agent rows: rows 4, 6, 7)

For every sub-agent registered on the device the catalogue carries its discover / invoke / chat / api-key triple plus any custom verbs. On this fixture device the sub-agents are `codex` and `web-builder`; with both mounted the rows look like:

<agent>.chat
The agent's main entry. Owner = agent/<u>.<agent>. This is what an LLM-routed user request hits. Without per-agent chat the only entry would be the generic OpenAI adapter (row 9), which is hub-routed — you could not address a specific sub-agent.

<agent>.discover
Per-agent ability catalogue filtered to this agent's visibility scope. The LLM uses it to enumerate "what verbs may I call as this agent" without ever having to read the device-wide catalogue.

<agent>.invoke
Generic Invoke entry-point for the agent: "call any ability you discovered as me". Without it, every per-verb invocation would need its own handler and the discover-then-invoke pattern would not compose.

<agent>.api_key.create, <agent>.api_key.list, <agent>.api_key.revoke
Agent-scoped API key management for the OpenAI-compat surface. Each agent owns its own key set. Today's catalogue still shows these under the daemon's pages-user identity (self.); the terminal form is per-agent.

web-builder.todo_add_task, web-builder.todo_delete_task, web-builder.todo_list_tasks
Custom verbs declared in the web-builder agent's workspace TOMLs. These are the canonical example of row 7 — per-agent custom abilities exposed through the same registry as the built-ins.

5.11 Agent-owned (LLM sub-agent surfaces): voice, meta, skill

These verbs live under web-builder's sub-agent in this fixture — they are re-exported per agent, not device-wide.

device.meta.describe, device.meta.list_abilities, device.meta.list_resources
Self-introspection over the seven-tuple. describe is the cheap identity+summary surface (federation peer hits this for cache checks); list_abilities returns the full descriptor catalogue; list_resources enumerates physical / logical resources (mics, cameras, files) the agent can name as subjects. Without these the LLM cannot know its own capabilities.

device.skill.list, device.skill.publish, device.skill.unpublish
Skill (curator-authored ability bundle) lifecycle. Publishing a skill mounts a new ability under the agent's namespace at runtime; this is how the alive-video, macos, etc. skills land on a device without redeploying.

device.voice.create_call, device.voice.join_call, device.voice.leave_call, device.voice.end_call, device.voice.list_calls, device.voice.show_call, device.voice.watch_call, device.voice.report_metrics
Voice/video call signaling. SDP, ICE, participant book-keeping. Without the signaling surface there is no protocol path for an agent to start or join a call — WebRTC needs an out-of-band rendezvous and these abilities provide it through Invoke.

device.voice.subscribe, device.voice.transcribe
Media planes: subscribe pushes TTS audio frames out; transcribe accepts streamed audio in and returns text. Together they make the agent voice-bidirectional.

5.12 Agent-owned — consent default helper (row 8 sub-case a)

Owner URA: easynet:///r/easynet.run/agent/test.consent-default-0.

device.consent.subscribe
Subscribe to the approval-broker pending queue. SnapshotThenLive stream; a UI joining mid-flight gets currently-pending requests + new ones.

device.consent.list_pending
One-shot snapshot of the same queue (the unary sibling).

device.consent.decide
Resolve a pending permission request by id; decision ∈ {allow, deny, deny_with_reason}.

Why these need a hosted helper agent (not just a daemon adapter). Consent decisions carry user-attributable signatures: the `deny` on a destructive Invoke is itself an Invoke, signed by the user (via the consent helper agent). A daemon-side adapter cannot sign as the user — only an agent the user owns can. Hence the helper-agent ownership form (row 8 sub-case a).

References

- `docs/rfc/AXON-RFC-001-plan-v4.1.5.md` — URA grammar §A.URA, AXIOM seven-tuple §9, ability registry §18.
- `docs/rfc/AXON-RFC-002-session-provider.md` — §3.3 device-bundle keyring (row 5).
- `docs/rfc/AXON-RFC-006-stateful-easynet.pdf` — §C OpenAI compatibility surface (row 9).
- `src/uri.rs` — canonical URA builders + parser; the §1 grammar is reflected here verbatim.
- `src/runtime/ability_descriptor.rs` — `AbilityDescriptor.owner_agent_uri`; v4.1.5 §9 `callee` constraint comment.
- `src/runtime/agents/meta_ability.rs` — `list_abilities_handler` synth path.
- `src/facade/cli/abilities.rs` — `easynet ability list` render layer.

End of specification.
Updates require an RFC bump.